04_placement 스크립트는 Synopsys의 **IC Compiler II (ICC2)** 환경에서 디자인의 셀 배치 및 최적화를 수행하는 **Placement (셀 배치)** 단계의 전체 Tcl 스크립트입니다.

앞선 파워플랜 데이터베이스를 넘겨받아 멀티 코너 멀티 모드(MCMM) 환경을 설정하고, 실제 타이밍 최적화를 곁들인 배치 공정(`place_opt`, `refine_opt`)을 거쳐 최종 검증 및 저장까지 수행하는 일련의 과정을 담고 있습니다.

---

1. 초기 실행 환경 및 타이머 설정

tcl

```
#made by kjinsoo
# module load
#module load icc2/2021.06-SP5
# open icc2_shell
#icc2_shell -gui | tee -i ./04_placement.log

#set current step & before step
set step placement
set before_step powerplan

#source design_setup.tcl
source -e ./icc2_common_env/design_setup.tcl

#source timer.rpt script - report runtime for placement
source ./icc2_common_env/sec_runtime.tcl

sec_StartTimer copy_lib

#set_host_options -max_cores 16
```

코드를 사용할 때는 주의가 필요합니다.

- 해설: 배치 단계 실행 로그(`04_placement.log`) 생성 지침 주석과 단계 변수(`step`, `before_step`) 선언부입니다.
- **`sec_runtime.tcl`** 및 **`sec_StartTimer copy_lib`**: 툴 내부 동작별 소요 시간(Runtime)을 정밀하게 기록하기 위한 커스텀 타이머 스크립트를 로드하고, 라이브러리 복사 구간의 측정을 시작합니다.
- **`set_host_options`**: 멀티 스레딩 속도 향상을 위해 최대 16개 코어를 할당하는 명령어(현재 주석 처리됨)입니다.

---

2. 라이브러리 입출력 및 연결성 바인딩

tcl

```
#open lib
open_lib ../03_powerplan/nlib/ORCA_${before_step}.nlib
#copy lib
copy_lib -from ../03_powerplan/nlib/ORCA_${before_step}.nlib -to
./nlib/ORCA_${step}.nlib
#close & open current lib
close_lib -all
open_lib ./nlib/ORCA_${step}.nlib
#open block
open_block ./nlib/ORCA_${step}.nlib:ORCA.design
#link_block
link_block -rebind

sec_ReportResourceUtil copy_lib
```

코드를 사용할 때는 주의가 필요합니다.

- 해설: 앞선 `powerplan` 라이브러리를 안전하게 가져와 현재 `placement` 경로에 새 라이브러리로 복사 및 오픈합니다.
- **`link_block -rebind`**: 현재 디자인 블록 내의 하이어라키 구조와 외부 공정 라이브러리(LEF/Frame 등) 간의 전기적/물리적 연결 관계를 새 데이터베이스 기준에 맞춰 다시 강하게 묶어주는 핵심 사전 작업입니다.
- **`sec_ReportResourceUtil copy_lib`**: 복사 및 로드 과정에 소요된 런타임과 메모리 자원 사용량을 리포트 파일에 기록합니다.

---

3. 라우팅 블로케이지 및 가이드 버퍼 삽입

tcl

```
#create routing blockages for all Macros & PLL
source -e ./user_script/create_routing_blockage.tcl

#insert guide buffer (all Macros & PLL signal inout port)
#insert buffer (scan_en_iopad/DOUT) - purpose of high_fanout_synthesis
source -e ./user_script/insert_guide_buf.tcl
```

코드를 사용할 때는 주의가 필요합니다.

- **`create_routing_blockage.tcl`**: 매크로(메모리)와 PLL 상단 및 주변 영역에 미세 신호선(Signal)이 무분별하게 지나가지 못하도록 라우팅 금지 구역(Routing Blockage)을 설정합니다. (크로스토크 및 노이즈 예방)
- **`insert_guide_buf.tcl`**: 매크로와 PLL의 입출력 신호 포트, 그리고 팬아웃(Fanout)이 매우 높을 것으로 예상되는 스캔

인에이블(`scan_en_iopad/DOUT`) 경로에 가이드 버퍼를 수동으로 삽입하여, 툴이 이후 배치 최적화 과정에서 구동력(Driving Strength) 부족 문제를 겪지 않도록 미리 가이드를 주는 단계입니다.

---

4. MCMM(Multi-Corner Multi-Mode) 환경 초기화 및 TLU+ 정보 로드

tcl

```
#remove scenarios modes corners before setting
remove_scenarios -all
remove_modes -all
remove_corners -all

# Read TLU+ files
read_parasitic_tech -name {minTLU} -tlup $data(tech_tlupbc)
read_parasitic_tech -name {maxTLU} -tlup $data(tech_tlupwc)

# source mcmm(multi corners multi modes).tcl
source -e ./icc2_common_env/mcmm.tcl
```

코드를 사용할 때는 주의가 필요합니다.

- **`remove_* -all`:** 기존에 누적되어 있던 시나리오, 동작 모드, 코너 설정을 깨끗하게 초기화합니다.
- **`read_parasitic_tech ...`:** 파운드리 사가 제공하는 **TLU+** 파일**(**금속 배선의 간격**/**두께별 정전용량 룩업 테이블**)**을 로드합니다. `minTLU`(Best Case/최소 커패시턴스)와 `maxTLU`(Worst Case/최대 커패시턴스)를 모두 읽어 들여 배치 단계에서 RC 배선 지연(Delay)을 매우 정확하게 예측할 수 있게 만듭니다.
- **`mcmm.tcl`:** 이 디자인이 직면할 다양한 전압, 온도, 주파수 동작 시나리오들의 뼈대를 구축합니다.

---

5. 동작 시나리오(Scenario Status) 활성화

tcl

```
# all about ss corner
set_scenario_status {*.ss*} -setup true -hold true \
                      -active true -max_transition true \
                      -max_capacitance true \
                      -min_capacitance false \
                        -leakage_power true \
                        -dynamic_power true
# all about ff corner
set_scenario_status {*.ff*} -setup true -hold true \
```

```
                                -active true -max_transition true \
                            -max_capacitance true \
                        -min_capacitance false \
                        -leakage_power true \
                             -dynamic_power true

#report_modes after set_scenario_status
report_modes
```

코드를 사용할 때는 주의가 필요합니다.

- 해설**:** 칩이 가장 느리게 동작하는 최악 조건의 코너(`*.ss*`)와 가장 빠르게 동작하는 최상 조건의 코너(`*.ff*`) 모두에 대해 **Setup/Hold** 타이밍 검사**,** 전력 최적화**(Leakage, Dynamic Power),** 그리고 최대 트랜지션 및 최대 커패시턴스 **DRC** 규칙을 한꺼번에 활성화**(**`true`)하여 툴이 이 모든 제약 조건을 만족하도록 유도합니다.

---

6. 모드별 SDC(제약 조건) 파일 로드 및 TIE 셀 목적 지정

tcl

```
#read_sdc(synopsys design constraints)
#scan_shift mode
current_mode scan_shift
read_sdc $data(shift_sdc) ;# ${libdir}/SDC/ORCA_shift.sdc

#func mode
current_mode func
read_sdc $data(func_sdc) ;# ${libdir}/SDC/ORCA_func.sdc

#scan_capture mode
current_mode scan_capture
read_sdc $data(capture_sdc) ;# ${libdir}/SDC/ORCA_capture.sdc

#set TIE cell
set_lib_cell_purpose -include optimization [get_lib_cells */TIE*] ;#
set TIE cell used in optimizaiton
set_dont_touch [get_lib_cells */TIE*] false ;# set TIE cells
"dont_touch" value false
```

코드를 사용할 때는 주의가 필요합니다.

- **`current_mode ... / read_sdc ...`:** 디자인의 세 가지 주요 동작 상태인 **1)** 테스트 데이터가 이동하는 모드**(`scan_shift`), 2)** 정상 동작 모드**(`func`), 3)**

테스트 결과를 래치하는 모드(`scan_capture`)를 각각 선언하고, 각 모드 전용의 타이밍 제약 조건 파일(SDC)을 완벽히 주입합니다.

- **`set_lib_cell_purpose ...`:** 논리 게이트 입력단에 고정 전압(High/Low)을 가해주는 TIE 셀들을 툴이 타이밍 및 DRC 최적화 과정에서 자유롭게 추가, 삭제, 변경할 수 있도록 최적화용(`optimization`)으로 지정하고 무조건 보호 설정(`dont_touch`)을 해제(`false`)합니다.

---

7. 배치 단계를 위한 클럭 가상화 (Ideal Network 설정)

tcl

```
#remove_propagated clocks before placement
#set ideal network for all clock
set mode "func scan_shift scan_capture"

foreach MODE $mode {
current_mode $MODE

echo "current mode is $MODE"
      switch $MODE {
            func {
                  remove_propagated_clocks [all_clocks]
                  set clock_sink [get_pins -hier -filter "is_clock_pin
== true || full_name == I_CLOCK_GEN/I_*/CLK_*X"]
                  remove_ideal_network $clock_sink
                  set_ideal_network [get_pins $clock_sink]
                  set_ideal_network [get_nets buf_sdram_clk]
            }
         ... (scan_shift 및 scan_capture 동일 구조로 생략) ...
      }
}
```

코드를 사용할 때는 주의가 필요합니다.

- **해설:** 아직 실제 클럭 트리 합성(CTS) 단계를 거치지 않았기 때문에, 현시점의 클럭 구조는 임시 가상 구조여야 합니다.
- 실제 메탈 배선을 타서 지연이 발생하는 프로파게이티드 속성을 제거(`remove_propagated_clocks`)하고, 전국의 모든 클럭 핀(`is_clock_pin == true`) 및 타겟 넷(`buf_sdram_clk`)을 지연 시간이 **0**인 완벽한 가상 네트워크(**`set_ideal_network`**)로 지정하여 배치 엔진이 데이터 신호선(Data Path) 자체의 최적화에만 오롯이 집중하도록 만드는 필수 세팅입니다.

---

8. 예외 타이밍 규칙(False Path) 설정 및 환경 로드

tcl

```
#set a false path constraint from the ScanEnable port (top-level)
current_mode "func"

#set false path
set_false_path -from scan_en -to [get_clocks "PCI* SD* SYS*"]

#report high fanout
report_net_fanout -high_fanout

current_mode "func"
#source set_disable_timing.tcl
source -e ./user_script/set_disable_timing.tcl
```

코드를 사용할 때는 주의가 필요합니다.

- **`set_false_path`**: 정상 동작(`func`) 모드에서는 스캔 테스트용 신호인 `scan_en`이 구동될 일이 없습니다. 따라서 이 핀에서 주 클럭 군(`PCI*, SD*, SYS*`)으로 가는 경로는 타이밍 계산에서 완전히 제외(**`False Path`**) 시켜 툴이 불필요한 계산에 자원을 낭비하지 않게 유도합니다.
- **`report_net_fanout`**: 수많은 셀을 구동하느라 과부하가 걸린 고-팬아웃 네트들의 목록을 확인합니다.
- **`set_disable_timing.tcl`**: 특정 로직 타이밍 아크를 끊어 불필요한 루프성 타이밍 계산을 방지하는 사용자 지정을 로드합니다.

---

9. 배치 엔진 세부 옵션(Application Options) 튜닝

tcl

```
#set_app_options for placement
set_app_options -list {opt.tie_cell.max_fanout {8}}
set_app_options -list {cts.common.max_fanout {16}}
set_app_options -list {opt.common.max_fanout {16}}
set_app_options -list {place.coarse.congestion_layer_aware {true}}
set_app_options -list {place.coarse.continue_on_missing_scandef {true}}
set_app_options -list {opt.common.drc_mode_buffering {true}}
set_app_options -list {place_opt.initial_drc.global_route_based {1}}
set_app_options -list {place_opt.initial_place.buffering_aware {true}}

set_app_options -list {place.coarse.pin_density_aware {true}}
set_app_options -list {place_opt.place.congestion_effort {high}}
set_app_options -list {place_opt.final_place.effort {high}}
```

코드를 사용할 때는 주의가 필요합니다.

- **`max_fanout {8 / 16}`**: TIE 셀 하나가 최대 8개 게이트까지만, 일반 신호선이나 CTS선은 최대 16개 게이트까지만 구동하도록 제약을 걸어 신호 무결성을 확보합니다.
- **`congestion_layer_aware`** / **`pin_density_aware`**: 메탈 레이어 특성 및 셀 핀 밀집도를 고려하여 배선이 꼬이고 막히는 정체 현상을 최소화하도록 엔진에 지시합니다.
- **`congestion_effort / effort {high}`**: 수만 개의 스탠다드 셀을 배치할 때 정체(Congestion) 완화 및 최종 타이밍 마진 확보를 위해 가장 연산 강도가 높은 고정밀 모드(`high`)를 기동합니다.

---

10. 1차 배치 및 최적화 실행 (`place_opt`)과 결과 리포트

tcl

```
#place_opt - place and optimize the current design
sec_StartTimer place_opt

#place_opt
place_opt

sec_ReportResourceUtil place_opt

#congestion
report_congestion -rerun_global_router ;#Reports the congestion
statistics
#timing
report_timing -significant_digits 5 ;#Displays timing information about
a design
#Qor summary
report_qor -summary ;#Displays QoR information and statistics for the
current design
```

코드를 사용할 때는 주의가 필요합니다.

- **`place_opt`**: 이 스크립트의 주인공급 명령어입니다. 가상의 글로벌 라우팅을 기반으로 초기 배치 **->** **DRC** 수정 **->** 초기 최적화 **->** 최종 배치 **->** 최종 최적화의 총 5단계를 거쳐 셀들을 최적의 위치에 실시간으로 심어 나갑니다.
- **`report_*`**: 배치가 끝난 뒤, 메탈선이 지나갈 자리가 부족하지 않은지(`congestion`), 타이밍 위반 수치는 소수점 5자리 기준으로 안전한지(`timing`), 전체적인 칩 퀄리티 지표(`qor`)는 어떤지 종합 성적표를 출력합니다.

---

11. 1차 배치 리포트 파일 추출 및 중간 저장

tcl

```
############################################################
#report Timing & constraints for placement after place_opt
############################################################
#update timing
update_timing -full ;#Updates timing information on the current design

sec_StartTimer report
#write "reports files" of placement at ${rundir}/reports/placement/*
directory
source ./user_script/placement_rpt.tcl
sec_ReportResourceUtil report

#save_lib
save_lib -all
```

코드를 사용할 때는 주의가 필요합니다.

- **`update_timing -full`**: 타이밍 그래프 계산을 전체 엔진 기준으로 완전히 동기화하여 최신 상태로 업데이트합니다.
- **`placement_rpt.tcl`**: 정밀 분석을 위해 지정된 폴더에 파일 형태로 다양한 스탯을 백업하고, 중간 결과 라이브러리를 통째로 하드디스크에 저장(`save_lib`)합니다.

---

12. 2차 정밀 재최적화 실행 (`refine_opt`) 및 스페어 셀 삽입

tcl

```
#refine_opt
set_app_options -list {refine_opt.place.congestion_effort {high}}
set_app_options -list {refine_opt.place.effort {high}}
set_app_options -list {place_opt.final_place.effort {high}}

sec_StartTimer refine_opt
refine_opt ;#By running refine_opt command, you can optimize the
place_opt completed db once again
sec_ReportResourceUtil refine_opt

#source Insert_spare cell script
source -e ./user_script/spare_cell_insertion.tcl
```

코드를 사용할 때는 주의가 필요합니다.

- **`refine_opt`:** 앞서 성공적으로 완료된 `place_opt` 결과물 기반 위에 한 번 더 강력한 알고리즘을 얹어 미세한 타이밍 마진과 혼잡도를 쥐어짜 내는 보정 단계입니다.
- **`spare_cell_insertion.tcl`:** 나중에 실리콘 웨이퍼가 구워진 후(Post-silicon) 미세한 논리 회로 수정이 필요할 때 메탈 레이어 마스크만 소폭 수정하여 칩을 고칠 수 있도록, 비어있는 빈 공간 군데군데 여분의 예비용 게이트 셀들을 의도적으로 심어두는 양산용 설계 테크닉입니다.

---

13. 최종 물리 법칙 및 합법성 검증 (Check Placement)

tcl

```
####################
#check placement
####################
#legality
check_legality ;#Checks the legality of the current placement

#mv_design
check_mv_design ;#Checks for violations in a multi-voltage design

#utilization of core area (use -region option)
report_utilization -region {{{425.5430 425.8460} {1014.4590
1014.1520}}} ;#Reports utilization of the current block

#congestion
report_congestion -rerun_global_router;#Reports the congestion
statistics
#timing
report_timing -significant_digits 5 ;#Displays timing information about
a design
#Qor summary
report_qor -summary ;#Displays QoR information and statistics for the
current design
```

코드를 사용할 때는 주의가 필요합니다.

- **`check_legality`:** 셀들이 격자(Site Row)를 벗어나 삐뚤어지거나, 서로 겹쳐서 배치(Overlap)되는 등 물리 법칙을 위반한 불법 배선 상태가 없는지 검증합니다.
- **`check_mv_design`:** 멀티 볼테이지(Multi-Voltage) 구조일 경우 전압 영역 전력 위반이 없는지 검사합니다.
- **`report_utilization -region`:** 지정한 핫스팟 중심 좌표 영역의 실질적인 셀 조밀도(Utilization)를 추출하여 과도하게 빽빽하지 않은지 모니터링합니다.

14. 최종 리포트 출력 및 정밀 배치 데이터베이스 별도 저장

tcl

```
#############################################################
#report Timing & constraints for placement after refine_opt
#############################################################
#update timing
update_timing -full ;#Updates timing information on the current design

sec_StartTimer report
#write "reports files" of placement at ${rundir}/reports/placement/*
directory
source ./user_script/refine_opt_rpt.tcl
sec_ReportResourceUtil report

#save_lib
save_lib -as ./nlib/ORCA_refine_opt.nlib
```

코드를 사용할 때는 주의가 필요합니다.

- 모든 재최적화 작업이 이상 없이 끝났으므로 최종 타이밍을 강제 업데이트하고 최종 파일 리포트(`refine_opt_rpt.tcl`)를 기록합니다.
- 마지막으로 다음 단계인 **Clock Tree Synthesis (CTS,** 클럭 트리 합성**)** 단계로 넘겨줄 완벽하게 최적화된 결과 데이터베이스를 `ORCA_refine_opt.nlib`라는 확정 명칭으로 따로 분리 저장하며 스크립트가 마무리됩니다.